We start out by creating our model class `wrapper`. This wrapper includes two functions, `feature_extractor` and `predict`. The `feature_extractor` function is required because otherwise, our fine-tuned model would not contain the same preprocessing step used during the fine-tuning and would, therefore, not be consistent during serving. Of course, you can simply serve your original model if you do not require any custom modifications to your model and only need the raw format outputs.

```
from io import BytesIO
import torchvision

class CVModelWrapper(mlflow.pyfunc.PythonModel):
    def __init__(self, model):
        # instantiate model in evaluation mode
        model.to(torch.device("cpu"))
        self.model = model.eval()

    def feature_extractor(self, image, p=0.5):…

    def predict(self, context, images):…
```

Figure 7.30 – Creating the model class wrapper

The `feature_extractor` function, which transforms an image into the format required by the served model, is the same code we used to score the model in *Chapter 6*. Let's dive into the prediction part, it's a similar function to what we have created in *Chapter 6* to score our model.

The `predict` function is similar to the one we used to score our model in *Chapter 6* using `pandas_udf`. Note that we are not returning only the predicted label but also a label corresponding to it in a dictionary format (this isn't required, but we wanted to show the output format's flexibility).